Comment définir

La phase **DÉFINIR** transforme les besoins élicités en **spécifications atomiques**, complètes, testables, modélisées et approuvées. Ces spécifications deviennent la source de vérité unique pour le scope, l'implémentation, l'intégration, la validation et l'automatisation.

> **Note terminologique.** Cette page décrit la phase **DÉFINIR** dans son contexte opérationnel. Pour les définitions de référence — **spécification IF… THEN…**, **spécification atomique**, **SPECIFY**, **traçabilité**, **DoR**, **DoD**, **validation**, **vérification**, **niveaux**, **interfaces**, **angles**, **rôles** et **SDLC** — voir le Lexique / Terminologies et la page **SPÉCIFICATION**. Les anciens termes comme critère d'acceptation, exigence, cas, scénario, Gherkin ou BDD convergent ici vers une seule nomenclature : la spécification IF… THEN….

> On ne peut pas bien implémenter ce qu'on n'a pas bien défini. Et on ne peut pas bien définir ce qu'on n'a pas bien élicité.

🎯 Objectif de la phase DÉFINIR

DÉFINIR est le cœur du SENSE : c'est ici que l'on passe du besoin compris au besoin **documenté, modélisé, validé et prêt à être réalisé**. La phase doit produire des spécifications suffisamment solides pour être utilisées par toutes les disciplines : produit, développement, QA, architecture, support et utilisateurs.

Le résultat attendu n'est pas une collection de documents séparés par silo, mais un **SPECIFY** cohérent, structuré et partagé, contenant des spécifications atomiques qui décrivent exactement ce qui doit être livré.

🔬 Sortie principale : des spécifications atomiques

Ce qui doit sortir de DÉFINIR, ce sont des **spécifications atomiques**. Une spécification atomique est assez petite pour être comprise, codée, intégrée et testée clairement, mais assez complète pour porter une valeur vérifiable.

Chaque spécification atomique sert simultanément de :

- **Scope de l'EPIC, release ou version** : ce qui est inclus, exclu et priorisé.
- **Contrat d'implémentation** : ce qui doit être codé, intégré ou modifié.
- **Contrat de validation** : ce qui doit être testé à tous les niveaux : small, medium, large / E2E.
- **Base d'automatisation** : ce qui pourra alimenter l'automatisation des tests, la génération assistée par IA et l'auto-validation.

> Si le dev code à partir d'un document et le QA teste à partir d'un autre, on a déjà perdu.

📘 Une seule documentation, utilisée par tous

La phase DÉFINIR doit produire une **source de vérité unique**. Il ne doit pas y avoir une documentation pour le PO, une autre pour le développeur, une autre pour le QA et une autre pour l'architecte. Tout le monde utilise les mêmes spécifications.

Rôle	Utilisation de la même spécification
PO / Product	Valide le scope, la valeur et l'alignement avec le besoin élicité.
Développeur	Implémente exactement ce qui est spécifié, avec les contraintes et impacts connus.
QA	Utilise les spécifications IF… THEN… comme source directe de validation, d'automatisation et de preuves, sans recréer une couche séparée de critères ou de scénarios.
Architecte	Valide les impacts systémiques, les dépendances et les contrats d'intégration.
Support / CARE	Comprend le comportement attendu pour diagnostiquer, confirmer et alimenter l'amélioration continue.
Utilisateur / spécialiste	Confirme que la spécification répond toujours au vrai besoin.

📐 Ce que DÉFINIR doit documenter

Dimension	Ce qu'il faut documenter sous forme de spécifications IF… THEN…

Fonctionnel	Comportements attendus, flux principaux et alternatifs, règles et résultats observables.
Non-fonctionnel	Performance, sécurité, compatibilité, résilience, utilisabilité, maintenabilité, avec seuils ou signaux mesurables.
Conditions limites	Entrées invalides, pertes réseau, erreurs, annulations, états concurrents, comportements dégradés.
Spécifications d'acceptation	Conditions IF… THEN… testables, mesurables, non ambiguës, avec résultats attendus. Ne pas recréer une couche séparée de critères d'acceptation.
Impacts systèmes	Dépendances inter-produits, inter-équipes, modules affectés, interfaces et intégrations touchées.
Traçabilité	Lien clair besoin → spécification → implémentation → preuve de validation.
Risques et dette technique	Risques connus, dette existante aggravée ou à résorber, impacts sur la qualité.
Estimation initiale	Estimation haut niveau au niveau EPIC / FEATURE pour soutenir la planification.

🔄 Concordance 100 % entre modèle, diagrammes et spécifications

Les diagrammes, les modèles et les flux de l'application doivent concorder à **100 %** avec les spécifications — et vice versa.

- Si un diagramme de séquence montre un flux, la spécification textuelle doit décrire le même flux.
- Si une spécification décrit une transition d'état, le modèle doit la représenter.
- Si une spécification IF… THEN… décrit un comportement attendu, le diagramme ou le modèle ne doit pas raconter autre chose.
- Toute divergence entre modèle, diagramme et texte est un défaut de définition à corriger avant PLANIFIER.

> Le modèle est la spécification visuelle. La spécification est le modèle textuel. Les deux doivent raconter exactement la même histoire.

🛠️ Techniques pour bien DÉFINIR

1. Spécification par l'exemple

Utiliser des exemples concrets pour rendre les comportements attendus testables. Le format commun est toujours une condition **IF… THEN…** :

- **IF** un contexte, une condition, une entrée, un état ou un événement est vrai
- **THEN** un comportement, résultat, contrainte, effet ou signal attendu doit être observable

Ces exemples deviennent des spécifications exécutables ou automatisables et réduisent l'ambiguïté entre PO, dev, QA et IA, sans maintenir un langage parallèle comme Gherkin, BDD ou des scénarios séparés.

2. Modélisation des flux et comportements

Utiliser des diagrammes de flux, des machines à états, des diagrammes de séquence ou des modèles de domaine pour représenter les comportements complexes. Les diagrammes ne sont pas décoratifs : ils font partie intégrante de la spécification.

3. Définition des cas limites et de la résilience

Pour chaque flux principal, se demander systématiquement : que se passe-t-il si le réseau tombe, si l'utilisateur annule, si deux événements arrivent en même temps, si une dépendance est indisponible ou si le système est dans un état inattendu ?

4. Analyse d'impact systémique

Identifier les modules, produits, équipes, contrats d'API, dépendances techniques et processus affectés. La question clé est : **si je change ce comportement, qui ou quoi est impacté ?**

5. Estimation initiale EPIC / FEATURE

Produire une estimation de haut niveau afin de soutenir la roadmap et la planification. À ce stade, on valide surtout la faisabilité, le scope et l'ordre de grandeur, avant le découpage détaillé en stories.

6. Identification des risques et de la dette technique

Documenter ce qui pourrait mal tourner, quelle dette technique influence la spécification, quelle dette est créée ou aggravée, et quelles mitigations doivent être considérées.

7. IA et automatisation dès la définition

Structurer les spécifications pour qu'elles soient utilisables par l'IA et l'automatisation : génération de cas de test, détection d'incohérences, assistance au codage, validation automatique et traçabilité.

🤖 Spécifications prêtes pour l'IA, l'automatisation et l'auto-validation

Les spécifications doivent être suffisamment solides, structurées et non ambiguës pour permettre l'utilisation de l'IA et de l'automatisation par la suite.

Levier	Ce que ça permet
Automatisation des validations	Générer ou dériver automatiquement des validations à partir des spécifications IF… THEN… et de leurs exemples.
Automatisation du codage	Utiliser l'IA pour générer du code conforme aux spécifications, aux modèles et aux contraintes définies.
Auto-validation	Vérifier automatiquement si l'implémentation respecte la spécification atomique.
Détection d'incohérences	Identifier les contradictions, ambiguïtés, trous de couverture et conditions limites manquantes.

Test de qualité d'une spécification : **est-ce qu'une IA pourrait générer le code et les tests à partir de cette spécification seule, sans poser de questions majeures ?** Si la réponse est non, la spécification n'est probablement pas assez solide.

🧪 Lien avec les niveaux de test

Chaque spécification atomique doit permettre de comprendre ce qui doit être testé à tous les niveaux pertinents.

Niveau	But	Ce que la spécification doit permettre
Small	Valider le plus petit comportement isolable.	Identifier les fonctions, règles ou unités logiques spécifiées et vérifiables.
Medium	Valider l'intégration entre composants.	Décrire les contrats, échanges, dépendances et interactions.
Large / E2E	Valider le comportement complet du point de vue utilisateur ou système.	Décrire les flux complets, états attendus, données et signaux observables.

Les approches whitebox, blackbox et graybox peuvent s'appliquer à ces niveaux. L'objectif de DÉFINIR est de rendre les comportements testables au bon niveau, pas de créer une documentation QA séparée.

📋 Contenu minimal d'un SPECIFY

Section	Contenu attendu
Description	Résumé clair du besoin et de la solution spécifiée.
Traçabilité	Liens vers les besoins, signaux, métriques ou sources issus d'ÉLICITER.
Spécifications atomiques	Liste des comportements à livrer, chacun formulé en IF... THEN... avec scope, règles, exemples et signaux observables.

Modèles et diagrammes	Flux, états, séquences ou modèles alignés à 100 % avec le texte.
Fonctionnel et non-fonctionnel	Comportements attendus, contraintes qualité, performance, sécurité, compatibilité, résilience.
Conditions limites	Erreurs, conditions aux frontières, comportements dégradés et états exceptionnels exprimés comme spécifications.
Spécifications d'acceptation	Spécifications mesurables, testables et reliées aux niveaux de validation, sans couche séparée de critères d'acceptation.
Impacts et dépendances	Systèmes, modules, équipes, interfaces et contrats affectés.
Risques et dette technique	Risques, dette identifiée, mitigations et décisions requises.
Automatisation	Stratégie de validation automatisée, possibilités d'IA, auto-validation et génération assistée.
Approbation	Validation par les parties prenantes, incluant la source, l'utilisateur ou le spécialiste lorsque nécessaire.

✅ Critères de sortie de DÉFINIR

La phase DÉFINIR est terminée lorsque :

- Les spécifications atomiques sont complètes, comprises et documentées.
- Le scope de l'EPIC, release ou version est clair.
- Ce qui doit être codé, intégré et livré est explicite.
- Ce qui doit être validé à tous les niveaux est explicite.
- Les spécifications IF... THEN... sont testables, mesurables et non ambiguës.
- Les impacts systèmes, dépendances et risques sont analysés et documentés.

- La dette technique est identifiée et rendue visible.
- La traçabilité besoin → spécification → implémentation → preuve de validation est assurée.
- Les diagrammes, modèles et spécifications concordent à 100 %.
- Les spécifications sont prêtes à soutenir l'IA, l'automatisation des validations, l'automatisation du codage et l'auto-validation.
- Une estimation initiale EPIC / FEATURE est réalisée.
- La documentation est centralisée, à jour et accessible.
- Le SPECIFY est validé et approuvé avant PLANIFIER / implémenter.
- Ce qui sera planifié et implémenté est revalidé avec l'utilisateur, directement ou indirectement, pour confirmer que cela fait toujours du sens.

🔁 Boucle de rétroaction vers ÉLICITER

DÉFINIR ne reçoit pas seulement l'information d'ÉLICITER : la phase doit aussi challenger ÉLICITER. Si, en définissant, l'équipe découvre que le besoin est incomplet, contradictoire, trop large, non mesurable ou déconnecté de la réalité utilisateur, l'information doit retourner à ÉLICITER sous forme de signaux, métriques, questions ou constats.

- Les incohérences deviennent des signaux à clarifier.
- Les métriques observées peuvent remettre en question le besoin initial.
- Les cas limites découverts peuvent enrichir l'élicitation.
- La validation utilisateur directe ou indirecte confirme que ce qui sera planifié et implémenté fait toujours du sens.

> Si en définissant on réalise que le besoin n'est pas clair, ce n'est pas un échec : c'est le système qui fonctionne. On retourne à ÉLICITER, on clarifie, et on revient.

👥 Qui fait quoi

Rôle	Contribution à DÉFINIR
Product Owner / Product Manager	Valide la valeur, le scope, les priorités et l'alignement avec le besoin élicité.
Développeur / Tech Lead	Valide la faisabilité technique, précise les contraintes

	d’implémentation et identifie les impacts sur le code.
QA / Testeur	S’assure que les spécifications IF… THEN… sont testables, observables et automatisables, sans dériver une nomenclature séparée de scénarios ou de cas.
Architecte	Analyse les impacts systémiques, les dépendances, les contrats d’intégration et les risques transversaux.
Support / CARE	Apporte les signaux terrain, les irritants récurrents et les besoins de diagnostic ou d’exploitabilité.
Utilisateur / spécialiste métier	Confirme que la spécification répond encore au besoin réel et que les exemples sont représentatifs.
Scrum Master / Coach	Facilite l’alignement entre les rôles et veille à ce que les ambiguïtés soient levées avant PLANIFIER.

📊 Métriques et signaux

Métrique / Signal	Ce qu’elle révèle	Source
Taux de spécifications retournées à ÉLICITER	Qualité de la compréhension initiale et nombre de besoins incomplets ou ambigus.	Jira / workflow SDLC
Nombre de spécifications IF… THEN… non testables	Niveau d’ambiguïté des spécifications et capacité à automatiser la validation.	Revue SPECIFY / QA

Nombre de divergences modèle ↔ texte ↔ spécification	Risque de désalignement entre documentation visuelle, fonctionnelle et validation.	Revue de spécification
Nombre de dépendances ou impacts non identifiés avant PLANIFIER	Maturité de l'analyse d'impact et risque de replanification tardive.	Architecture / planning
Dette technique identifiée pendant DÉFINIR	Risque qualité ou maintenabilité à intégrer dans le scope, les compromis ou les mitigations.	SonarQube / revues techniques
Complexité estimée par spécification atomique	Capacité à découper, prioriser et planifier sans regrouper des comportements trop larges.	Atelier DÉFINIR / estimation

⚠️ Pièges courants à éviter

Piège	Pourquoi c'est un problème	Comment l'éviter
Spécifier en silo	Chaque rôle crée sa propre interprétation.	Utiliser une seule documentation et une seule spécification partagée.
Spécifications vagues	Impossible de valider ou d'automatiser correctement.	Formuler des spécifications IF... THEN... observables, mesurables et reliées à des exemples.

Modèle différent de la spec	Le texte et les diagrammes racontent deux histoires différentes.	Valider systématiquement la concordance modèle ↔ texte ↔ spécification.
Ignorer les conditions limites	Les défauts apparaissent souvent dans les comportements non spécifiés.	Explorer les erreurs, états limites, pertes de dépendances et comportements dégradés.
Commencer à coder sans SPECIFY approuvé	Le travail démarre avant que la cible soit claire.	Exiger une validation de DÉFINIR avant PLANIFIER / implémenter.
Ne pas challenger ÉLICITER	On planifie un besoin qui n'est peut-être plus clair ou pertinent.	Retourner les signaux, métriques et questions à ÉLICITER, puis revalider avec l'utilisateur.
Spécifications non exploitables par l'IA	L'automatisation produit des résultats incomplets ou incohérents.	Structurer les spécifications, exemples et modèles de façon non ambiguë.

🔄 Lien avec les autres phases

```
ÉLICITER → besoins compris → DÉFINIR → spécifications atomiques approuvées → PLANIFIER
    ↑                                  ↓
    └──── signaux, métriques, questions, validation utilisateur ────┘
```

DÉFINIR est le pont entre le besoin et la réalisation. Quand cette phase est solide, PLANIFIER, CODER, INTÉGRER, TESTER et CONFIRMER s'alignent tous sur la même vérité : les spécifications atomiques.

Références

- SPÉCIFICATION
- Exemples concrets

- Les tests (niveau × interface × angle × type)
- Cérémonie de définition
- Comment éliciter — phase précédente
- Comment planifier — phase suivante
- (SDLC) Software Development Life Cycle (officiel)
- Contrat qualité officiel OPAL-RT
- RACI — Rôles et responsabilités
- Qualimétrie officielle OPAL-RT
- Recommandations — enjeux, constats ou lacunes à résoudre / solutionner